ULModule3Assignment

October 11, 2026

[49]:
```
# First import some necessary libraries
import numpy as np
import pandas as pd
up, down = True, False
np.set_printoptions(precision=3)
from sklearn.feature_extraction.text import TfidfVectorizer

import pandas as pd
import gzip
from sklearn import decomposition
from sklearn.decomposition import TruncatedSVD
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
from sklearn.decomposition import LatentDirichletAllocation
```

All import statements are provided for you- please do not import other libraries in your own code.

1 Assignment 3: Text Representations, Topic Modeling, and Word Embeddings

For the Module 3 programming assignment, you will focus on topic modeling using a dataset of recipes. You will be doing a number of tasks pertaining to topic modeling, such as obtaining top topic terms with Non-negative Matrix Factorization (NMF) and using Latent Semantic Indexing to do semantic comparison of recipe ingredient lists.

In modern-day applications, topic modeling can be very useful when attempting to understand themes/subjects across a large number of documents.

- First, you are going to compute topic models using non-negative matrix factorization. You'll write a function to return the top cluster terms for each of ten topics found by NMF on the recipe dataset.
- Second, you are going to use Latent Semantic Indexing (LSI) to use a model trained from the recipes dataset to find the ingredient lists most and least similar to a given single ingredient list.
- Third, you are going to explore how to estimate semantic coherence of a topic, using a provided

word embedding to compute the semantic similarity of a topic's terms. In the first task, you created a list of top topic terms for the first 10 topics in the recipe dataset derived using NMF. Now, you are going to use a pre-trained word embedding to estimate the semantic coherence of those topics from task 1.

- Fourth, you will explore the fascinating phenomenon of 'semantic arithmetic' that has been observed in common word embeddings, using the provided word embedding trained from Amazon review data.

[50]:
```
# Preamble code providing vectorizer and reading datasets.

def parse(path, limit=None):
    g = gzip.open(path, 'rb')
    for i, l in enumerate(g):
        if limit is not None and i >= limit:
            break
        yield eval(l)

def getDF(path, limit=None):
    df = {}
    for i, d in enumerate(parse(path, limit)):
        df[i] = d
    return pd.DataFrame.from_dict(df, orient='index')

#See topics.
#Pick 2 different categories and compare topics coming out of each.
df = getDF('./assets/reviews_Electronics_5.json.gz', limit=3000)
reviews_series = df['reviewText']
recipes_df = pd.read_csv("./assets/RAW_recipes_processed.csv")
ingredients_series = recipes_df["ingredients"]


tfidf_vectorizer = TfidfVectorizer(
    max_features=10000,  # only top 10k by freq
    lowercase=False,  # keep capitalization
    ngram_range=(1, 2),  # include 2-word phrases
    min_df=10,  # note: absolute count of doc
    max_df=0.95,  # note: % of docs
    stop_words="english",
)  # default English stopwords
```

2 Part 1: Topic modeling using Non-negative Matrix Factorization

(25 points)

For the first task, you are going to compute topic models using non-negative matrix factorization (NMF).

Your function should compute an NMF topic model on the `reviews_series` data using 10 topics (`n_components=10`). Then, for each topic found by NMF, your code should return the list of the 8 highest-weighted terms in that topic.

For this function:

1. Loop through indexes 0-9 (inclusive) across the NMF topics
2. For each topic, get the indices of the top terms. `argsort()` and `nmf.components_` are useful here.
3. Within this loop, for the current topic, loop through the indices of top terms: for each term, look up the LOWERCASE version of the term string in the `stop_words` set, and ignore any terms that are in that stopword set. This is so we don't consider very common words like "the" or "and" as part of the topic model. Stop processing the topic when you have added 8 terms to the current topic term list.
4. Append the resulting list of top topic terms to `top_words_per_topic` and continue looping to the next topic.
5. Return the list of 10 top term lists.

[51]:
```
task_id = "1"
```

[52]:
```
def task_1_solution():
    #DO NOT MODIFY THIS CODE BLOCK
    X = tfidf_vectorizer.fit_transform(reviews_series)
    tfidf_feature_names = tfidf_vectorizer.get_feature_names_out()
    n_topics = 10
    nmf = decomposition.NMF(n_components=n_topics, random_state=0,␣
 ↪init="nndsvd")
    nmf.fit_transform(X)
    top = 8
    # Load English stopwords
    stop_words = set(ENGLISH_STOP_WORDS)

    top_words_per_topic = []
    # YOUR CODE HERE
    #raise NotImplementedError()
    for i in range(n_topics):
        #print(tfidf_feature_names[np.argsort(-nmf.components_[i])])
        #top_words_per_topic.append(list(set(map(lambda x: x.lower(),␣
 ↪tfidf_feature_names[np.argsort(-nmf.components_[i])])) - set(stop_words)))
        top_words = []
        for word in tfidf_feature_names[np.argsort(-nmf.components_[i])]:
            if not word.lower() in stop_words:
                top_words.append(word)
                if len(top_words) >= 8:
                    break
        top_words_per_topic.append(top_words)

    return top_words_per_topic
```

[53]:
```
# Use this cell to explore your solution

task_1_solution()
```

[53]:
```
[['tapes', 'player', 'CD', 'DVD', 'tape', 'VCR', 'product', 'VHS'],
 ['Nook', 'books', 'Kindle', 'screen', 'read', 'nook', 'book', 'tablet'],
 ['cable',
  'This cable',
  'cable works',
  'router',
  'The cable',
  'Belkin',
  'needed',
  'ethernet'],
 ['mount', 'TV', 'wall', '34', 'tv', 'install', '32', 'wall mount'],
 ['drive', 'floppy', 'USB', 'disks', 'old', 'disk', 'Windows', 'data'],
 ['Palm', 'case', 'quot', 'stylus', 'Vx', 'hard', 'cradle', 'plastic'],
 ['works',
  'great',
  'works great',
  'It works',
  'fine',
  'works fine',
  'product',
  'does'],
 ['radio', 'sound', 'reception', 'good', 'FM', 'stations', 'shower', 'radios'],
 ['cables', 'good', 'price', 'cord', 'work', 'Belkin', 'quality', 'great'],
 ['camera',
  'pictures',
  'use',
  'batteries',
  'card',
  'digital',
  'charger',
  'quot']]
```

[8]:
```
print(f"Task {task_id} - AG tests")
stu_ans = task_1_solution()


print(f"Task {task_id} - your answer:\n{stu_ans}")

assert isinstance(
    stu_ans, list
), f"Task {task_id}: Your function should return a list of lists "
```

```
del stu_ans
```

```
Task 1 - AG tests
Task 1 - your answer:
[['tapes', 'player', 'CD', 'DVD', 'tape', 'VCR', 'product', 'VHS'], ['Nook',
'books', 'Kindle', 'screen', 'read', 'The', 'nook', 'book'], ['cable', 'This
cable', 'cable works', 'router', 'The cable', 'Belkin', 'needed', 'ethernet'],
['mount', 'TV', 'wall', '34', 'tv', 'install', '32', 'wall mount'], ['drive',
'floppy', 'USB', 'disks', 'old', 'disk', 'Windows', 'data'], ['Palm', 'case',
'quot', 'The', 'stylus', 'Vx', 'hard', 'cradle'], ['works', 'great', 'It',
'works great', 'It works', 'fine', 'This', 'works fine'], ['radio', 'sound',
'AM', 'reception', 'good', 'The', 'FM', 'stations'], ['cables', 'good', 'price',
'cord', 'work', 'Belkin', 'quality', 'They'], ['camera', 'pictures', 'use',
'batteries', 'card', 'digital', 'charger', 'The']]
```

3 Task 2a: Latent Semantic Indexing (LSI)

(10 points)

In Module 1 when discussing applications of Singular Value Decomposition, you saw an example that applied SVD to text called Latent Semantic Indexing (LSI). We're going to explore that idea in this task since it can be regarded as a form of topic modeling.

In this task, we have a ingredient list for a meal named `single_ingredient_list`, and a list of related ingredient lists defined below stored in `related_ingredient_lists`.

Our goal for this task will be to use LSI to find which ingredient lists in `related_ingredient_list` are most and least similar to the one we have in `single_ingredient_list`. To do this, your code is going to use LSI to convert each ingredient list to a vector in LSI space, and then compare the LSI vectors with cosine similarity.

[9]:
```
### Run this preamble code to run LSI. We've also given the line of code that␣
 ↪gets the resulting U matrix.
tfidf_vectorizer = TfidfVectorizer(
    ngram_range=(1, 1), min_df=2, max_df=0.95, stop_words="english",␣
 ↪max_features=10000
)  # default English stopwords
tfidf_documents = tfidf_vectorizer.fit_transform(ingredients_series)
# .get_feature_names() is deprecated in 1.0
# tfidf_feature_names = tfidf_vectorizer.get_feature_names()
tfidf_feature_names = tfidf_vectorizer.get_feature_names_out()
# LSI does truncated SVD on the document-term matrix of tf.idf term-weights.
# The matrix we got back from the vectorizer is a
# document-term matrix, i.e. one row per document.
n_topics = 200
lsi = TruncatedSVD(n_components=n_topics, random_state=0)
```

```
# To match the examples and development of LSI in
# our lectures, we're going to
# take the transpose of the document-term matrix to give
# TruncatedSVD the term-document matrix as input.
# This is the matrix U_k:  num_term_features x num_topics
reduced_term_matrix = lsi.fit_transform(np.transpose(tfidf_documents))


single_ingredient_list = "Chicken breast, Broccoli, Carrots, Rice, Soy sauce"

# Related Ingredient Lists
related_ingredient_lists = [
    "Beef sirloin, Spinach, Potatoes, Quinoa, Balsamic vinegar",
    "Salmon fillet, Asparagus, Sweet potatoes, Couscous, Lemon juice",
    "Pork tenderloin, Green beans, Brown rice, Teriyaki sauce, Scallions",
    "Shrimp, Bell peppers, Corn, Black beans, Lime juice",
    "Tofu, Snap peas, Cauliflower, Quinoa, Tamari"
]
```

In regards to computing the similarity between two text objects(strings) we must first learn how to convert an ingredient list into an LSI-expanded TF-IDF representation of the list. For this conversion we use the formula:

$q_k = q \cdot (U_k \Sigma_k^{-1})$

With this formula, you'll "expand" both the query and the document vectors to add related terms.

Let's walk through these steps (**Note** that in matrix math, dimensions and hence order of operations matters!).
1. The `reduced_term_matrix` (which we have defined for you above) corresponds to the LSI matrix U_k, and this is *multiplied* (* operator) with the inverse of matrix Σ_k, the `LSI.singular_values_` matrix. **Note** that Σ_k is raised to the power of negative one Σ_k^{-1}. The reason we invert Σ_k is that there is no division operation with matrices so we invert and multiply! - Think of this step as forming a normalized scaler for the LSI latent factor weights (the 'topics').
- For this step, you'll need `.dot()`, `np.linalg.inv`, and `np.diag` in numpy.

2. The vectorized query matrix q (or document d) we store in `query_vector` is then dotted (dot-product) with the result of $U_k \Sigma_k^{-1}$.

The resulting q_k vector is now an LSI-expanded representation of your original ingredient list `q` in LSI space, where we are going to do our vector cosine similarity comparisons.

[10]:
```
#For lsi, compare two different things

task_id = '2a'



#make 2b: least similar ingredient list and score
```

[11]:
```
def task_2a_solution():
    query_vector = tfidf_vectorizer.transform([single_ingredient_list])
    query_vector_lsi = query_vector.dot(reduced_term_matrix.dot(np.diag(1/lsi.
 ↪singular_values_)))

    # YOUR CODE HERE
    #raise NotImplementedError()

    return query_vector_lsi
```

[12]:
```
# Use this cell to explore your solution.

task_2a_solution()
```

[12]:
```
array([[ 0.106,  0.171, -0.096, -0.079, -0.202,  0.233, -0.166,  0.039,
        -0.023, -0.03 ,  0.24 ,  0.132,  0.026, -0.032, -0.046, -0.037,
         0.005, -0.019, -0.024, -0.019,  0.029,  0.012,  0.046, -0.037,
        -0.021, -0.007,  0.003, -0.067, -0.043, -0.016,  0.007, -0.028,
         0.036,  0.025,  0.008, -0.048, -0.012,  0.02 , -0.048, -0.082,
        -0.02 , -0.04 , -0.005,  0.002,  0.018, -0.026, -0.06 , -0.007,
         0.014,  0.015, -0.074, -0.025, -0.047, -0.038, -0.05 ,  0.034,
        -0.013, -0.047,  0.047, -0.004, -0.01 , -0.035, -0.019, -0.012,
         0.045, -0.015,  0.01 ,  0.016, -0.013, -0.001,  0.029, -0.001,
        -0.011, -0.034,  0.024, -0.013, -0.075, -0.01 , -0.064, -0.022,
         0.057,  0.001,  0.018,  0.012, -0.049, -0.008,  0.008, -0.047,
         0.022, -0.078, -0.01 ,  0.028,  0.05 , -0.015,  0.086, -0.031,
         0.004, -0.074,  0.113,  0.03 ,  0.001,  0.049, -0.029,  0.023,
         0.018, -0.092,  0.097, -0.029,  0.039, -0.032, -0.093,  0.119,
        -0.01 , -0.118, -0.032, -0.14 , -0.101,  0.05 ,  0.065, -0.016,
         0.06 ,  0.023,  0.061, -0.102,  0.115, -0.029,  0.038, -0.012,
        -0.017,  0.067, -0.002,  0.017, -0.002, -0.017,  0.036,  0.015,
        -0.016,  0.072,  0.009, -0.058, -0.002,  0.053,  0.077,  0.101,
        -0.08 , -0.051,  0.004,  0.012, -0.023, -0.014,  0.005, -0.055,
        -0.03 ,  0.111, -0.001, -0.005,  0.021,  0.17 , -0.02 ,  0.011,
        -0.039, -0.046,  0.092, -0.059,  0.027, -0.073, -0.013, -0.056,
         0.045,  0.075, -0.022,  0.049, -0.132,  0.053, -0.033, -0.036,
         0.017,  0.093, -0.004,  0.029, -0.028,  0.015,  0.005, -0.062,
         0.028, -0.014,  0.137,  0.07 ,  0.02 , -0.032,  0.02 ,  0.01 ,
         0.035, -0.004, -0.016,  0.001,  0.03 , -0.035,  0.015, -0.009]])
```

[13]:
```
print(f"Task {task_id} - AG tests")
stu_ans = task_2a_solution()


print(f"Task {task_id} - your answer:\n{stu_ans}")

assert isinstance(
```

```
    stu_ans, np.ndarray
), f"Task {task_id}: Your function should return an ndarray "



del stu_ans
```

```
Task 2a - AG tests
Task 2a - your answer:
[[ 0.106  0.171 -0.096 -0.079 -0.202  0.233 -0.166  0.039 -0.023 -0.03
   0.24   0.132  0.026 -0.032 -0.046 -0.037  0.005 -0.019 -0.024 -0.019
   0.029  0.012  0.046 -0.037 -0.021 -0.007  0.003 -0.067 -0.043 -0.016
   0.007 -0.028  0.036  0.025  0.008 -0.048 -0.012  0.02  -0.048 -0.082
  -0.02  -0.04  -0.005  0.002  0.018 -0.026 -0.06  -0.007  0.014  0.015
  -0.074 -0.025 -0.047 -0.038 -0.05   0.034 -0.013 -0.047  0.047 -0.004
  -0.01  -0.035 -0.019 -0.012  0.045 -0.015  0.01   0.016 -0.013 -0.001
   0.029 -0.001 -0.011 -0.034  0.024 -0.013 -0.075 -0.01  -0.064 -0.022
   0.057  0.001  0.018  0.012 -0.049 -0.008  0.008 -0.047  0.022 -0.078
  -0.01   0.028  0.05  -0.015  0.086 -0.031  0.004 -0.074  0.113  0.03
   0.001  0.049 -0.029  0.023  0.018 -0.092  0.097 -0.029  0.039 -0.032
  -0.093  0.119 -0.01  -0.118 -0.032 -0.14  -0.101  0.05   0.065 -0.016
   0.06   0.023  0.061 -0.102  0.115 -0.029  0.038 -0.012 -0.017  0.067
  -0.002  0.017 -0.002 -0.017  0.036  0.015 -0.016  0.072  0.009 -0.058
  -0.002  0.053  0.077  0.101 -0.08  -0.051  0.004  0.012 -0.023 -0.014
   0.005 -0.055 -0.03   0.111 -0.001 -0.005  0.021  0.17  -0.02   0.011
  -0.039 -0.046  0.092 -0.059  0.027 -0.073 -0.013 -0.056  0.045  0.075
  -0.022  0.049 -0.132  0.053 -0.033 -0.036  0.017  0.093 -0.004  0.029
  -0.028  0.015  0.005 -0.062  0.028 -0.014  0.137  0.07   0.02  -0.032
   0.02   0.01   0.035 -0.004 -0.016  0.001  0.03  -0.035  0.015 -0.009]]
```

4 Task 2b. Semantic similarity using LSI

(15 points)

From task 2a, you now know how to calculate an LSI-expanded version of an ingredient list. Now, it's time to calculate the cosine similarity between our query vector `single_ingredient_list` and those in `related_ingredient_lists` to find the most similar list in `related_ingredient_list`.

Calculating cosine similarity is easy: to compute it between two LSI-expanded lists, use the following code:

```
cos_similarity_object = cosine_similarity(query_vector_lsi, related_vector_lsi)
cos_similarity_score = cos_similarity_object[0][0].
```

With the knowledge you now have, you can use simple loops to loop through the list of related ingredient lists and obtain the most similar ingredient lists, along with the similarity scores. The format of how they should be returned is already defined for you.

`task_2b_solution` should return the cosine similarity score of the most similar ingredient list, as well as that most similar list itself.

[14]:
```
task_id = '2b'
```

[15]:
```
def task_2b_solution():
    query_vector_lsi = task_2a_solution()


    max_cos_lsi = 0.0
    most_similar_list = []

    # YOUR CODE HERE
    #raise NotImplementedError()
    for related_ingredient_list in related_ingredient_lists:
        related_vector = tfidf_vectorizer.transform([related_ingredient_list])
        related_vector_lsi = related_vector.dot(reduced_term_matrix.dot(np.
 ↪diag(1/lsi.singular_values_)))
        score = cosine_similarity(query_vector_lsi, related_vector_lsi)[0][0]
        if max_cos_lsi <= score:
            max_cos_lsi, most_similar_list = score, related_ingredient_list

    return max_cos_lsi, most_similar_list
```

[16]:
```
# Use this cell to explore your solution.

task_2b_solution()
```

[16]:
```
(0.2715288064675456,
 'Pork tenderloin, Green beans, Brown rice, Teriyaki sauce, Scallions')
```

[17]:
```
print(f"Task {task_id} - AG tests")
stu_ans = task_2b_solution()


print(f"Task {task_id} - your answer:\n{stu_ans}")

assert isinstance(
    stu_ans, tuple
), f"Task {task_id}: Your function should return a tuple "

assert isinstance(
    stu_ans[0], float
), f"Task {task_id}: max_cos_lsi should be a float "

assert isinstance(
    stu_ans[1], str
```

```
), f"Task {task_id}: most_similar_list should be a str "
```

```
Task 2b - AG tests
Task 2b - your answer:
(0.2715288064675456, 'Pork tenderloin, Green beans, Brown rice, Teriyaki sauce,
Scallions')
```

5 Task 2c: Semantic similarity using LSI: least similar items

(15 points)

Now, instead of finding the most similar list with the highest cosine similarity, find the least similar list with the smallest cosine similarity.

[18]:
```
task_id = '2c'
```

[19]:
```
def task_2c_solution():
    query_vector_lsi = task_2a_solution()


    min_cos_lsi = 0.0
    least_similar_list = []

    # YOUR CODE HERE
    #raise NotImplementedError()
    for related_ingredient_list in related_ingredient_lists:
        related_vector = tfidf_vectorizer.transform([related_ingredient_list])
        related_vector_lsi = related_vector.dot(reduced_term_matrix.dot(np.
 ↪diag(1/lsi.singular_values_)))
        score = cosine_similarity(query_vector_lsi, related_vector_lsi)[0][0]
        if min_cos_lsi >= score:
            min_cos_lsi, least_similar_list = score, related_ingredient_list

    return min_cos_lsi, least_similar_list
```

[20]:
```
# Use this cell to explore your solution.

task_2c_solution()
```

[20]:
```
(-0.04085776840509708,
 'Beef sirloin, Spinach, Potatoes, Quinoa, Balsamic vinegar')
```

[21]:
```
print(f"Task {task_id} - AG tests")
stu_ans = task_2c_solution()


print(f"Task {task_id} - your answer:\n{stu_ans}")
```

```
assert isinstance(
    stu_ans, tuple
), f"Task {task_id}: Your function should return a tuple "

assert isinstance(
    stu_ans[0], float
), f"Task {task_id}: max_cos_lsi should be a float "

assert isinstance(
    stu_ans[1], str
), f"Task {task_id}: most_similar_list should be a str "
```

```
Task 2c - AG tests
Task 2c - your answer:
(-0.04085776840509708, 'Beef sirloin, Spinach, Potatoes, Quinoa, Balsamic
vinegar')
```

[22]:
```
#This might take a minute or so to run! It will be relevant for the next part.
word_embeddings = {}
with open("./assets/review_qa_300d.vec", 'r', encoding='utf-8') as f:
    # Skip the first line containing metadata
    next(f)
    for line in f:
        # Split the line by whitespace
        parts = line.strip().split(' ')
        word = parts[0]  # Get the word
        embedding_vector = np.array([float(x) for x in parts[1:]])
        word_embeddings[word] = embedding_vector
```

6 Task 3: Optimizing topic coherency using word embeddings

(15 points)

Semantically coherent topics, where the terms are related to each other, are more desirable because they are easier to interpret and do a better job of capturing a common theme. In contrast, topics whose terms appear randomly selected are less desirable. So for this task, we're going to explore how to select more coherent topic models by using an embedding to compute text similarity.

In task 1, you created a list of top topic terms for the first 10 topics in the recipe dataset's NMF object. Here, we're going to compute a coherence score for those topics, by providing you a calculation that uses a word embedding to compute semantically-based word similarities. With these similarities, we will be able to estimate a coherence measure for individual topics. (If we wanted, we could then compute an overall topic model coherence score by combining the individual topic coherence scores, e.g. by taking the mean coherence score across individual topics) (The embedding we use is derived from a large set of Amazon reviews.)

Your solution should loop through every topic obtained from your task_1_solution() and use

the utility function `topical_coherence_from_vec_file` provided below to compute the coherence score for each topic's set of terms. Your function should return a tuple with two items: (a) the top topic terms list that obtained the maximum coherence score (as computed by `topical_coherence_from_vec_file`) and (b) the corresponding coherence score obtained for that top topic.

[23]:
```
#Topic coherence: Use existing embedding to compute text similarity

def topical_coherence_from_vec_file(terms):
    # Initialize an empty list to store word embedding vectors
    embedding_vectors = []
    # Extract embedding vectors for the given terms
    for term in terms:
        if term in word_embeddings:
            embedding_vectors.append(word_embeddings[term])
    # If no embedding vectors were found for the terms, return coherence of 0
    if len(embedding_vectors) == 0:
        return 0.0
    # Compute pairwise cosine similarity between embedding vectors
    similarity_matrix = cosine_similarity(embedding_vectors)
    # Set diagonal elements to zero to ignore self-similarities
    np.fill_diagonal(similarity_matrix, 0)
    # Compute the mean over all pairwise cosine similarities
    coherence = np.mean(similarity_matrix)

    return coherence
```

[24]:
```
task_id = 3
```

[27]:
```
def task_3_solution():
    review_list = task_1_solution()
    most_similar = []
    max_coherence = 0
    # YOUR CODE HERE
    #raise NotImplementedError()
    for topic in review_list:
        c = topical_coherence_from_vec_file(topic)
        if c >= max_coherence:
            max_coherence, most_similar = c, topic
    return (most_similar, max_coherence)
```

[28]:
```
# Use this cell to explore your solution.

task_3_solution()
```

[28]:
```
(['player', 'tapes', 'cd', 'dvd', 'vcr', 'tape', 'cleaner', 'vhs'],
 0.4010120794672034)
```

[29]:
```
print(f"Task {task_id} - AG tests")
stu_ans = task_3_solution()


print(f"Task {task_id} - your answer:\n{stu_ans}")

assert isinstance(
    stu_ans, tuple
), f"Task {task_id}: Your function should return a tuple "

assert isinstance(
    stu_ans[0], list
), f"Task {task_id}: The first element in the tuple should be a 1x8 list."




del stu_ans
```

```
Task 3 - AG tests
Task 3 - your answer:
(['player', 'tapes', 'cd', 'dvd', 'vcr', 'tape', 'cleaner', 'vhs'],
0.4010120794672034)
```

7 Task 4: Semantic arithmetic with embeddings

(20 points)

Many word-based embeddings have an interesting quality: their vectors often satisfy arithmetic relations that reflect semantic addition and subtraction with the underlying meaning. For example, if you have embedding vectors for the words 'king', 'man' and 'woman', in many popular embeddings if you compute the arithmetic combination King - Man + Woman, the resulting vector would be very close to the embedding for 'queen'. In some sense, King - Man captures the 'royalty' aspect of a king that can be transferred to other entities. This is easy to experiment with in Python, and that will be the basis of this task.

We're going to test this by seeing which words in an embedding are closest to the embedding vector representing the arithmetic combination Laptop - Keyboard + Touch.

Your code will accomplish this by looping through word and vector in the `word_embeddings.items()` attribute. Within the loop, select words that have *all* of the following requirements:

- the distance between the current word vector and the `vec_desired_word` vector given in the provided code is greater than zero AND less than 4.3. Recall that the 'vec_desired_word" vector represents the vector computed using Laptop - Keyboard + Touch.
- the current word is alphanumeric (use `isalnum()` to test)
- the current word is not in stop_words (test to see if in `stop_words` set)
- the current word is not 'laptop', 'keyword', or 'touch'.

- stop when you have added 10 words to the result list.

You should notice the word in the list at index 6 is very appropriate for laptop - keyboard + touch!

[31]:
```
stop_words = set(ENGLISH_STOP_WORDS)

task_id = 4
```

[46]:
```
def task_4_solution():
    word1 = 'laptop'
    word2 = 'keyboard'
    word3 = 'touch'
    min_distance = 4.3
    vec_word1 = word_embeddings[word1]
    vec_word2 = word_embeddings[word2]
    vec_word3 = word_embeddings[word3]
    vec_desired_word = vec_word1 - vec_word2 + vec_word3
    result = []
    #DO NOT ALTER THE ABOVE CODE.

    # YOUR CODE HERE
    #raise NotImplementedError()

    for word, wordvec in word_embeddings.items():
        if word.isalnum() and not word in stop_words and word not in [word1,␣
 ↪word2, word3]:
            #d = np.sqrt(np.sum((vec_desired_word - wordvec)**2))
            d = np.linalg.norm(vec_desired_word - wordvec)
            if d > 0 and d < min_distance:
                result.append(word)
                if len(result) >= 10:
                    break

    return result
```

[47]:
```
# Use this cell to explore your solution.

task_4_solution()
```

[47]:
```
['computer',
 'actually',
 'laptops',
 'notebook',
 'touched',
 'touches',
 'itouch',
 'touchable',
 'especally',
```

```
 'mycomputer']
```

[48]:
```
print(f"Task {task_id} - AG tests")
stu_ans = task_4_solution()


print(f"Task {task_id} - your answer:\n{stu_ans}")


assert isinstance(
    stu_ans, list
), f"Task {task_id}: Your function should return a list. "

assert (
    len(stu_ans) == 10
), f"Task {task_id}: Your function should return an list of length 10. "


# Some hidden tests

del stu_ans
```

```
Task 4 - AG tests
Task 4 - your answer:
['computer', 'actually', 'laptops', 'notebook', 'touched', 'touches', 'itouch',
'touchable', 'especally', 'mycomputer']
```

[ ]:
```

```